

PROGRAMMATION LANGUAGE PHP

FORMATEUR :

Samuel KOTEI-NIKOI,

Ingénieur de Conception en Informatique,

Consultant en gestion des systèmes d'informations,

Directeur Général NIKKOSA COMMUNICATION,

Fondateur du portail www.abidjan-immobilier.net,

Contacts : +225 07 35 47 28

PARTIE 1 : LES BASES DU PHP

- I. Introduction à PHP
- II. Préparer son ordinateur
- III. Premiers pas avec PHP
- IV. Inclure des portions de page
- V. Les variables
- VI. Les conditions
- VII. Les boucles
- VIII. Les fonctions
- IX. Les tableaux

I : Introduction à PHP

1. Les sites statiques et dynamiques

On considère qu'il existe deux types de sites web : les sites **statiques** et les sites **dynamiques**.

Les sites statiques : ce sont des sites réalisés uniquement à l'aide des langages HTML et CSS. Ils fonctionnent très bien mais leur contenu ne peut pas être mis à jour automatiquement : il faut que le propriétaire du site (le webmaster) modifie le code source pour y ajouter des nouveautés. Ce n'est pas très pratique quand on doit mettre à jour son site plusieurs fois dans la même journée ! Les sites statiques sont donc bien adaptés pour réaliser des sites « vitrine », pour présenter par exemple son entreprise, mais sans aller plus loin.

Les sites dynamiques : plus complexes, ils utilisent d'autres langages en plus de HTML et CSS, tels que PHP et MySQL. Le contenu de ces sites web est dit « dynamique » parce qu'il peut changer sans l'intervention du webmaster !

Le seul prérequis pour apprendre à créer ce type de sites est de déjà savoir réaliser des sites statiques en HTML et CSS.

2. Comment fonctionne un site web ?

Lorsque vous voulez visiter un site web, vous tapez son adresse dans votre navigateur web : Mozilla Firefox, Internet Explorer, Opera, Safari... . Mais comment fait la page web pour arriver jusqu'à vous ?

Il faut savoir qu'Internet est un réseau composé d'ordinateurs. Ceux-ci peuvent être classés en deux catégories.

Les clients : ce sont les ordinateurs des internautes comme vous. Votre ordinateur fait donc partie de la catégorie des clients. Chaque client représente un visiteur d'un site web.

Les serveurs : ce sont des ordinateurs puissants qui stockent et délivrent des sites web aux internautes, c'est-à-dire aux clients. Les serveurs sont indispensables au bon fonctionnement du Web.

En résumé, votre ordinateur est appelé **le client**, tandis que l'ordinateur qui détient le site web est appelé **le serveur**. Leur manière de communiquer est ce qui fait la différence entre un site statique et un site dynamique.

Cas d'un site statique

Lorsque le site est statique, le schéma est très simple. Cela se passe en deux temps :

1. le client demande au serveur à voir une page web ;
2. le serveur lui répond en lui envoyant la page réclamée.



Cas d'un site dynamique

Lorsque le site est dynamique, il y a une étape intermédiaire : la page est **générée**

1. Le client demande au serveur à voir une page web ;
2. le serveur prépare la page spécialement pour le client ;
3. le serveur lui envoie la page qu'il vient de générer.



3. Les langages du web

Lorsqu'on crée un site web, on est amené à manipuler non pas un mais plusieurs langages. En tant que webmaster, il faut impérativement les connaître.

Pour un site statique : HTML et CSS

HTML : c'est le langage à la base des sites web. Simple à apprendre, il fonctionne à partir de balises.

CSS : c'est le langage de mise en forme des sites web. Alors que le HTML permet d'écrire le contenu de vos pages web et de les structurer, le langage CSS s'occupe de la mise en forme et de la mise en page.

Ces langages sont la base de tous les sites web. Lorsque le serveur envoie la page web au client, il envoie en fait du code en langage HTML et CSS.

Le problème, c'est que lorsqu'on connaît **seulement** HTML et CSS, on ne peut produire que des sites statiques... et non des sites dynamiques ! Pour ces derniers, il est nécessaire de manipuler d'autres langages en plus de HTML et CSS.

Pour un site dynamique : ajoutez PHP et MySQL

PHP : c'est un langage que seuls les serveurs comprennent et qui permet de rendre votre site dynamique. C'est PHP qui « génère » la page web comme on l'a vu sur un des schémas précédents.

Ce sera le premier langage que nous découvrirons dans ce cours. Il peut fonctionner seul, mais il ne prend vraiment de l'intérêt que s'il est combiné à un outil tel que MySQL.

MySQL : c'est ce qu'on appelle un SGBD (Système de Gestion de Base de Données). Pour faire simple, son rôle est d'enregistrer des données de manière organisée afin de vous aider à les retrouver facilement plus tard. C'est grâce à MySQL que vous pourrez enregistrer la liste des membres de votre site, les messages postés sur le forum, etc. Le langage qui permet de communiquer avec la base de données s'appelle le SQL.

Les autres langages

HTML et CSS n'ont pas de concurrents car ce sont des standards. Tout le monde est censé les connaître et les utiliser sur tous les sites web.

En revanche, pour ce qui est des sites dynamiques, PHP et MySQL sont loin d'être les seuls : ASP.NET, JAVA, RUBY on RAILS, ORACLE, MICROSOFT SQL SERVER, POSTGRESQL...

II : Préparer son ordinateur

De quels programmes a-t-on besoin ?

Selon que l'on crée un site statique ou un site dynamique, on a besoin de logiciels différents.

Avec un site statique

Un éditeur de texte : en théorie, un programme tel que le Bloc-notes livré avec Windows suffit, bien qu'il soit recommandé d'utiliser un outil un peu plus évolué comme Notepad++ ou ADOBE DREAMWEAVER.

Un navigateur web : il permet de tester la page web. Vous pouvez utiliser par exemple Mozilla Firefox, Internet Explorer, Google Chrome, Opera, Safari, ou tout autre navigateur auquel vous êtes habitués pour aller sur le web. Il est conseillé de tester son site régulièrement sur différents navigateurs.

Cependant, pour la création de sites internet dynamiques, ces outils ne suffisent pas. Il est nécessaire d'installer des programmes supplémentaires.

Avec un site dynamique

Pour que votre ordinateur puisse lire du PHP, il faut qu'il se comporte comme un serveur. Rassurez-vous, vous n'avez pas besoin d'acheter une machine spéciale pour cela : il suffit simplement d'installer les mêmes programmes que ceux que l'on trouve sur les serveurs qui délivrent les sites web aux internautes.

Apache : c'est ce qu'on appelle un serveur web. Il s'agit du plus important de tous les programmes, car c'est lui qui est chargé de délivrer les pages web aux visiteurs. Cependant, Apache ne gère que les sites web statiques (il ne peut traiter que des pages HTML). Il faut donc le compléter avec d'autres programmes.

PHP : c'est un plug-in pour Apache qui le rend capable de traiter des pages web dynamiques en PHP. En clair, en combinant Apache et PHP, notre ordinateur sera capable de lire des pages web en PHP.

MySQL : c'est le logiciel de gestion de bases de données. Il permet d'enregistrer des données de manière organisée.

Il existe plusieurs paquetages tout prêts pour Windows. Nous utiliserons WAMP Server qui a l'avantage d'être régulièrement mis à jour et disponible en français.

III : Premiers pas en PHP

1. Les balises PHP

La forme d'une balise PHP

Pour utiliser du PHP, on il faut introduire une nouvelle balise un peu spéciale. Elle commence par `<?php` et se termine par `?>` ; c'est à l'intérieur que l'on mettra du code source PHP.

Exemple :

```
<?php
```

```
    /* Le code PHP se met ici
```

```
    Et ici
```

```
    Et encore ici */
```

```
?>
```

Insérer une balise PHP au milieu du code HTML

La balise PHP précédente s'insère au milieu du code HTML. Elle peut s'insérer n'importe où dans la page : dans l'entête, dans le corps, au milieu d'une balise HTML.

2. Afficher du texte

L'instruction echo

Tout langage de programmation contient ce qu'on appelle des **instructions**. On en écrit une par ligne en général, et elles se terminent toutes par un point-virgule. Une instruction commande à l'ordinateur d'effectuer une action précise.

Ici, la première instruction que nous allons découvrir permet d'insérer du texte dans la page web. Il s'agit de l'instruction **echo**, la plus simple et la plus basique de toutes les instructions que vous devez connaître.

Voici un exemple d'utilisation de cette instruction :

```
<?php  
    echo "Ceci est du texte";  
?>
```

Comme vous le voyez, à l'intérieur de la balise PHP on écrit l'instruction echo suivie du texte à afficher entre guillemets. Les guillemets permettent de délimiter le début et la fin du texte, ce qui aide l'ordinateur à se repérer. Enfin, l'instruction se termine par un point-virgule, ce qui signifie **Fin de l'instruction**.

Il faut savoir qu'on a aussi le droit de demander d'afficher des balises. Par exemple, le code suivant fonctionne :

```
<?php  
    echo "Ceci est du <strong>texte</strong>";  
?>
```

Comment faire pour afficher un guillemet ?

Si vous mettez un guillemet, ça veut dire pour l'ordinateur que le texte à afficher s'arrête là. Vous risquez au mieux de faire planter votre code et d'avoir une « Parse error ». La solution consiste à faire précéder le guillemet d'un antislash \ :

```
<?php  
    echo "Cette ligne a été écrite \"uniquement\" en PHP.";  
?>
```

3. Les commentaires

Un **commentaire** est un texte que vous mettez pour vous dans le code PHP. Ce texte est ignoré, c'est-à-dire qu'il disparaît complètement lors de la génération de la page. Il n'y a que vous qui voyez ce texte. Cela permet de vous retrouver dans votre code PHP, parce que si vous n'y touchez pas pendant des semaines et que vous y revenez, vous risquez d'être un peu perdus. Vous pouvez écrire tout et n'importe quoi, le tout est de s'en servir à bon escient.

Il existe deux types de commentaires :

- les commentaires monolignes ;
- les commentaires multilignes.

Tout dépend de la longueur de votre commentaire.

Les commentaires monolignes

Pour indiquer que vous écrivez un commentaire sur une seule ligne, vous devez taper deux slashes : « **//** ». Tapez ensuite votre commentaire.

Exemple :

```
<?php
```

```
    echo "J'habite en Chine."; // Cette ligne indique où j'habite
```

```
    // La ligne suivante indique mon âge
```

```
    echo "J'ai 92 ans.";
```

```
?>
```

Nous avons dans cet exemple deux commentaires à des endroits différents :

- le premier est à la fin d'une ligne ;
- le second est sur toute une ligne.

À vous de voir où vous placez vos commentaires : si vous commentez une ligne précise, mieux vaut mettre le commentaire à la fin de cette ligne.

Les commentaires multilignes

Ce sont les plus pratiques si vous pensez écrire un commentaire sur plusieurs lignes, mais on peut aussi s'en servir pour écrire des commentaires d'une seule ligne. Il faut commencer par écrire `/*` puis refermer par `*/`.

Exemple :

```
<?php
    /* La ligne suivante indique mon âge
    Si vous ne me croyez pas...
    ... vous avez raison ;*/
    echo "J'ai 92 ans.";
?>
```

IV : Inclure des portions de page

Le principe

La plupart des sites web sont généralement découpés selon le schéma suivant.



Le problème

Jusqu'ici, vous étiez condamnés à copier sur chaque page à l'identique :

- l'en-tête ;
- le menu ;
- le pied de page.

Cela donnait du code lourd et répétitif sur toutes les pages !

La solution

En PHP, nous pouvons facilement insérer d'autres pages (on peut aussi insérer seulement des morceaux de pages) à l'intérieur d'une page.

Le principe de fonctionnement des **inclusions** en PHP est plutôt simple à comprendre. Vous avez un site web composé de disons vingt pages. Sur chaque page, il y a un menu, toujours le même. Pourquoi ne pas écrire ce menu (et seulement lui) une seule fois dans une page menus.php ?

En PHP, vous allez pouvoir inclure votre menu sur toutes vos pages. Lorsque vous voudrez modifier votre menu, vous n'aurez qu'à modifier menus.php et l'ensemble des pages de votre site web sera automatiquement mis à jour !

L'instruction pour inclure le contenu d'une page est :

`include ("chemin_d'accès_fichier");`

Exemples :

```
<?php include("menus.php"); ?>
```

```
<?php include("includes/menus.php"); ?>
```


V : Les variables

1. Qu'est ce qu'une variable ?

Une **variable**, c'est une petite information stockée en mémoire **temporairement**. Elle n'a pas une grande durée de vie. En PHP, la variable (l'information) existe tant que la page est en cours de génération. Dès que la page PHP est générée, toutes les variables sont supprimées de la mémoire car elles ne servent plus à rien. Ce n'est donc pas un fichier qui reste stocké sur le disque dur mais une petite information temporaire présente en mémoire vive. C'est à vous de créer des variables. Vous en créez quand vous en avez besoin pour retenir des informations.

Un nom et une valeur

Une variable est toujours constituée de deux éléments :

son nom : pour pouvoir la reconnaître, vous devez donner un nom à votre variable. Par exemple `age_du_visiteur` ;

sa valeur : c'est l'information qu'elle contient, et qui peut changer. Par exemple : 17. Dans cet exemple nous avons une variable appelée `age_du_visiteur` qui a pour valeur 17. On peut modifier quand on veut la valeur de cette variable, faire des opérations dessus, etc. Et quand on en a besoin, on l'appelle (par son nom), et elle nous dit la valeur qu'elle contient.

Les différents types de variables

Les variables sont capables de stocker différents types d'informations. On parle de **types de données**. Voici les principaux types à connaître.

Les chaînes de caractères (string) : les **chaînes de caractères** sont le nom informatique qu'on donne au texte. Tout texte est appelé chaîne de caractères. En PHP, ce type de données a un nom : string. On peut stocker des textes courts comme très longs au besoin.

Les nombres entiers (int) : ce sont les nombres du type 1, 2, 3, 4, etc. On compte aussi parmi eux les entiers relatifs : -1, -2, -3...

Les nombres décimaux (float) : ce sont les nombres à virgule, comme 14,738. On peut stocker de nombreux chiffres après la virgule, ce qui devrait convenir pour la plupart des usages que vous en ferez. Attention, les nombres doivent être écrits avec un point au lieu de la virgule (c'est la notation anglaise).

Les booléens (bool) : c'est un type très important qui permet de stocker soit vrai soit faux. Cela permet de retenir si une information est vraie ou fausse. On les utilise très fréquemment. On écrit **true** pour vrai, et **false** pour faux.

Rien (NULL) : aussi bizarre que cela puisse paraître, on a parfois besoin de dire qu'une variable ne contient rien. Rien du tout. On indique donc qu'elle vaut NULL. Ce n'est pas vraiment un type de données, mais plutôt l'**absence** de type.

2. Affecter une valeur à une variable

Premières manipulations de variables

Commençons par regarder le code ci-dessous :

```
<?php
    $age_du_visiteur = 17;
?>
```

Avec ce code PHP, on vient de créer une variable :

- son nom est `age_du_visiteur` ;
- sa valeur est 17.

Notez qu'on ne peut pas mettre d'espace dans un nom de variable. À la place, on utilise un *underscore* « `_` ».

Pour le nom, évitez aussi les accents, les cédilles et tout autre symbole.

Analysons dans le détail le code qu'on vient de voir.

- D'abord, on écrit le symbole « dollar » (\$) : il précède toujours le nom d'une variable.
- Ensuite, il y a le signe « égal » (=) : celui-là c'est logique, c'est pour dire que `$age_du_visiteur` est égal à...
- À la suite, il y a la valeur de la variable, ici 17.
- Enfin, il y a le point-virgule (;) qui permet de terminer l'instruction.

Utilisation des types de données

Le type string (chaîne de caractères)

Ce type permet de stocker du texte. Pour cela, vous devez entourer votre texte de guillemets doubles `""` ou de guillemets simples `' '` (attention, ce sont des apostrophes).

Exemple :

```
<?php
    $variable = "Mon \"nom\" est Mateo21"; //Remarquez l'utilisation de l'antislash
    $variable = 'Je m\'appelle Mateo21'; //Remarquez l'utilisation de l'antislash
?>
```

Le type int (nombre entier)

Il suffit tout simplement d'écrire le nombre que vous voulez stocker, sans guillemets.

```
<?php
    $age_du_visiteur = 17;
?>
```

Le type float (nombre décimal)

Vous devez écrire votre nombre avec un point au lieu d'une virgule.

```
<?php
    $poids = 57.3;
?>
```

Le type bool (booléen)

Pour dire si une variable vaut vrai ou faux, vous devez écrire le mot **true** ou **false** sans guillemets autour (ce n'est pas une chaîne de caractères). Il faut bien choisir le nom de votre variable pour que l'on comprenne ce que ça signifie.

Exemple :

```
<?php
    $je_suis_un_apprenant = true;
    $je_suis_bon_en_php = false;
?>
```

Une variable vide avec NULL

Si vous voulez créer une variable qui ne contient rien, vous devez lui passer le mot-clé NULL (vous pouvez aussi l'écrire en minuscules : null).

```
<?php
    $pas_de_valeur = NULL;
?>
```

Cela sert simplement à indiquer que la variable ne contient rien, tout du moins pour le moment.

3. Afficher et concaténer des variables

Afficher le contenu d'une variable

Pour afficher la valeur d'une variable on utilise l'instruction **echo** suivie du nom de la variable.

Exemple :

```
<?php
    $age_du_visiteur = 17;
    echo $age_du_visiteur;
?>
```

Comme vous le voyez, il suffit d'écrire le nom de la variable que vous voulez afficher.

La concaténation

Cela signifie **assemblage**. La concaténation est un moyen d'assembler du texte et des variables.

Pour cela il faut utiliser des guillemets simples et écrire la variable en dehors des guillemets et séparer les éléments les uns des autres à l'aide d'un point.

Exemple :

```
<?php
    $age_du_visiteur = 17;
    echo 'Le visiteur a ' . $age_du_visiteur . ' ans';
?>
```

4. Faire des calculs simples

Les opérations de base : addition, soustraction...

Les signes à connaître pour faire les quatre opérations de base sont :

- +** : l'addition
- : la soustraction
- *** : la multiplication
- /** : la division

Exemple :

```
<?php
```

```
$nombre = 2 + 4; // $nombre prend la valeur 6
```

```
$nombre = 5 - 1; // $nombre prend la valeur 4
```

```
$nombre = 3 * 5; // $nombre prend la valeur 15
```

```
$nombre = 10 / 2; // $nombre prend la valeur 5
```

```
// Allez on rajoute un peu de difficulté
```

```
$nombre = 3 * 5 + 1; // $nombre prend la valeur 16
```

```
$nombre = (1 + 2) * 2; // $nombre prend la valeur 6
```

```
?>
```

Le modulo

Il est possible de faire un autre type d'opération un peu moins connu : le **modulo**. Cela représente le reste de la division entière.

Par exemple, $6 / 3 = 2$ et il n'y a pas de reste. En revanche, $7 / 3 = 2$ (car le nombre 3 « rentre » 2 fois dans le nombre 7) et il reste 1.

Le modulo permet justement de récupérer ce « reste ». Pour faire un calcul avec un modulo, on utilise le symbole %.

Exemple :

```
<?php
```

```
    $nombre = 10 % 5; // $nombre prend la valeur 0 car la division tombe juste
```

```
    $nombre = 10 % 3; // $nombre prend la valeur 1 car il reste 1
```

```
?>
```

Et les autres opérations ?

Nous passerons sous silence les opérations plus complexes telles que la racine carrée, l'exponentielle, la factorielle, etc. Toutes ces opérations peuvent être réalisées en PHP mais il faudra passer par ce qu'on appelle des fonctions, une notion que l'on découvrira plus tard. Les opérations basiques que l'on vient de voir sont amplement suffisantes pour la programmation PHP de tous les jours.

VI : Les conditions

Ce chapitre est d'une importance capitale. En effet, vous serez très souvent amenés à employer des **conditions** dans vos pages web PHP.

À quoi servent les conditions ? On a parfois besoin d'afficher des choses différentes en fonction de certaines données. Par exemple, si c'est le matin, vous voudrez dire « Bonjour » à votre visiteur ; si c'est le soir, il vaudra mieux dire « Bonsoir ».

C'est là qu'interviennent les conditions. Elles permettent de donner des ordres différents à PHP selon le cas. Pour notre exemple, on lui dirait : *Si c'est le matin, affiche « Bonjour ».* *Sinon, si c'est le soir, affiche « Bonsoir ».* Vous allez voir que les conditions constituent vraiment la base pour rendre votre site dynamique, c'est-à-dire pour afficher des choses **différentes** en fonction du visiteur, de la date, de l'heure de la journée, etc.

Voilà pourquoi ce chapitre est si important !

1. La structure de base : if...else

Une condition peut être écrite en PHP sous différentes formes. On parle de structures **conditionnelles**.

Pour étudier la structure **if... else**, nous allons suivre le plan qui suit :

- **Les symboles à connaître** : il va d'abord falloir retenir quelques symboles qui permettent de faire des comparaisons. Soyez attentifs car ils vous seront utiles pour les conditions.
- **La structure if... else** : c'est le gros morceau. Là vous allez voir comment fonctionne une condition avec if... else. Inutile de vous dire qu'il est indispensable de bien comprendre cela.
- **Des conditions multiples** : on compliquera un peu nos conditions. Vous allez voir en effet qu'on peut utiliser plusieurs conditions à la fois.
- **Le cas des booléens** : nous verrons ensuite qu'il existe une façon particulière d'utiliser les conditions quand on travaille sur des booléens. Si vous ne savez pas ce que sont les booléens, revoyez le chapitre sur les variables.

Les symboles à connaître

Juste avant de commencer, il est important de connaître les symboles que nous serons amenés à utiliser. Voici ci-dessous un tableau avec ces symboles et leur signification.

Symbole	Signification
==	Est égal à
>	Est supérieur à
<	Est inférieur à
>=	Est supérieur ou égal à
<=	Est inférieur ou égal à
!=	Est différent de

NB : Il y a deux symboles « égal » (==) sur la première ligne, et il ne faut pas confondre ça avec le simple = que nous avons appris dans le chapitre sur les variables. Ici, le double égal sert à tester l'égalité, à dire « Si c'est égal à... ».

Dans les conditions, on utilisera toujours le double égal (==).

La structure if... else

Voici ce qu'on doit écrire, dans l'ordre, pour utiliser une condition.

Pour introduire une condition, on utilise le mot **if**, qui en anglais signifie « si ».

On ajoute à la suite **entre parenthèses** la condition en elle-même (vous allez voir que vous pouvez inventer une infinité de conditions).

Enfin, **on ouvre des accolades** à l'intérieur desquelles on placera les instructions à exécuter si la condition est remplie.

Exemple :

```
<?php
    $age = 8;
    if ($age <= 12)
    {
        echo "Salut gamin !";
    }
?>
```

Ici, on demande à PHP : **si la variable \$age est inférieure ou égale à 12, affiche « Salut gamin ! ».**

Vous remarquerez que dans la quasi-totalité des cas, c'est sur une variable qu'on fait la condition.

Dans notre exemple, on travaille sur la variable \$age. Ce qui compte ici, c'est qu'il y a deux possibilités : soit la condition est remplie (l'âge est inférieur ou égal à 12 ans) et alors on affiche quelque chose ; sinon, on saute les instructions entre accolades, on ne fait rien,

On peut maintenant améliorer notre exemple. On va afficher un autre message si l'âge est supérieur à 12 ans :

```
<?php
    $age = 8;

    if ($age <= 12) // Si l'âge est inférieur ou égal à 12
    {
        echo "Salut gamin ! Bienvenue sur mon site !<br />";
        $autorisation_entrer = "Oui";
    }
    else // SINON
    {
        echo "Ceci est un site pour enfants, vous êtes trop vieux pour pouvoir entrer. Au r
        evoir !<br />";
        $autorisation_entrer = "Non";
    }

    echo "Avez-vous l'autorisation d'entrer ? La réponse est : $autorisation_entrer";
?>
```

On peut toujours améliorer notre code en ajoutant ceci :

```
<?php
    if ($autorisation_entrer == "Oui") // SI on a l'autorisation d'entrer
    {
        // instructions à exécuter quand on est autorisé à entrer
    }
    elseif ($autorisation_entrer == "Non") // SINON SI on n'a pas l'autorisation d'entrer
    {
        // instructions à exécuter quand on n'est pas autorisé à entrer
    }
    else // SINON (la variable ne contient ni Oui ni Non, on ne peut pas agir)
    {
        echo "Euh, je ne connais pas ton âge, tu peux me le rappeler s'il te plaît ?";
    }
?>
```

La principale nouveauté ici, c'est le mot-clé **elseif** qui signifie « sinon si ». Dans l'ordre, PHP rencontre les conditions suivantes :

- si \$autorisation_entrer est égale à « Oui », tu exécutes ces instructions...
- sinon si \$autorisation_entrer est égale à « Non », tu exécutes ces autres instructions...
- sinon, tu redemandes l'âge pour savoir si on a ou non l'autorisation d'entrer.

Le cas des booléens

Si vous regardez bien le dernier code source (avec \$autorisation_entrer), vous ne trouvez pas qu'il serait plus adapté d'utiliser des booléens ?

Ce sont ces variables qui valent soit **true** (vrai) soit **false** (faux). Eh bien, les booléens sont particulièrement utiles avec les conditions ! Voici comment on teste une variable booléenne :

```
<?php
    if ($autorisation_entrer == true)
    {
        echo "Bienvenue petit . .)";
    }
    elseif ($autorisation_entrer == false)
    {
        echo "T'as pas le droit d'entrer !";
    }
?>
```

Des conditions multiples

Ce qu'on va essayer de faire, c'est de poser plusieurs conditions à la fois. Pour cela, on aura besoin de nouveaux mots-clés. Voici les principaux à connaître :

Mot-clé	Signification	Symbole équivalent
AND	Et	&&
OR	Ou	

Exemple d'utilisation :

```
<?php
    if ($age <= 12 AND $sexe == "garçon")
    {
        echo "Bienvenue sur le site des garçons!";
    }
    elseif ($age <= 12 AND $sexe == "fille")
    {
        echo "C'est pas un site pour les filles ici, retourne jouer à la Barbie !";
    }
?>
```


C'est tout simple en fait et ça se comprend très bien : si l'âge est inférieur ou égal à 12 ans et que c'est un garçon, on lui permet d'accéder au site des garçons.
Sinon, si c'est une fille dont l'âge est inférieur ou égal à 12 ans, on l'oblige à sortir du site.

Un dernier exemple avec **OR** :

```
<?php
    if ($sexe == "fille" OR $sexe == "garçon")
    {
        echo "Salut Terrien !";
    }
    else
    {
        echo "Euh, si t'es ni une fille ni un garçon, t'es quoi alors ?";
    }
?>
```

2. Une alternative pratique : switch

En théorie, les structures à base de if... elseif... else que nous venons de voir suffisent pour traiter n'importe quelle condition. Cependant il existe une autre structure mieux adaptée dans certains cas. Voyons l'exemple ci-dessous :

```
<?php
```

```
    if ($note == 0)
    {
        echo "Tu es vraiment un gros Zéro !!!";
    }
    elseif ($note == 5)
    {
        echo "Tu es très mauvais";
    }
    elseif ($note == 7)
    {
        echo "Tu es mauvais";
    }
    elseif ($note == 10)
    {
        echo "Tu as pile poil la moyenne, c'est un peu juste...";
    }
```

```
elseif ($note == 12)
{
    echo "Tu es assez bon";
}
elseif ($note == 16)
{
    echo "Tu te débrouilles très bien !";
}
elseif ($note == 20)
{
    echo "Excellent travail, c'est parfait !";
}
else
{
    echo "Désolé, je n'ai pas de message à afficher pour cette note";
}

?>
```

Comme vous le voyez, c'est lourd, long, et répétitif. Dans ce cas, on peut utiliser une autre structure plus souple : c'est switch.

Voici le même exemple avec switch (le résultat est le même, mais le code est plus adapté) :

<?php

```
$note = 10;
switch ($note) // on indique sur quelle variable on travaille
{
    case 0: // dans le cas où $note vaut 0
        echo "Tu es vraiment nul !!!";
        break;
    case 5: // dans le cas où $note vaut 5
        echo "Tu es très mauvais";
        break;
    case 7: // dans le cas où $note vaut 7
        echo "Tu es mauvais";
        break;
    case 10: echo "Tu as tout juste la moyenne...";
        break;
    case 12: echo "Tu es assez bon";
        break;
    case 16: echo "Tu te débrouilles très bien !";
        break;
    case 20: echo "Excellent travail, c'est parfait !";
        break;
    default: echo "Désolé, je n'ai pas de message à afficher pour cette note";
}
```

?>

En observant ce code nous constatons les différences.

Tout d'abord, il y a beaucoup moins d'accolades (elles marquent seulement le début et la fin du **switch**).

case signifie « cas ». Dans le switch, on indique au début sur quelle variable on travaille (ici \$note). On dit à PHP : **Je vais analyser la valeur de \$note**. Après, on utilise des case pour analyser chaque cas (case 0, case 10, etc.). Cela signifie : **Dans le cas où la valeur est 0...**

Dans le cas où la valeur est 10...

Avantage : on n'a plus besoin de mettre le double égal ! Défaut : ça ne marche pas avec les autres symboles (< > <= >= !=). En clair, le switch ne peut tester que l'égalité.

Le mot-clé **default** à la fin est un peu l'équivalent du else. C'est le message qui s'affiche par défaut quelle que soit la valeur de la variable.

Il y a cependant une chose importante à savoir : supposons dans notre exemple que la note soit de 10. PHP va lire : case 0 ? Non. Je saute. case 5 ? Non plus. Je saute. case 7 ? Non plus. Je saute. Case 10 ? Oui, j'exécute les instructions. Mais contrairement aux elseif, PHP ne s'arrête pas là et continue à lire les instructions des case qui suivent ! case 12, case 16, etc. Pour empêcher cela, utilisez l'instruction break;. L'instruction break demande à PHP de sortir du switch. Dès que PHP tombe sur break, il sort des accolades et donc il ne lit pas les case qui suivent. En pratique, on utilise très souvent un break car sinon, PHP lit des instructions qui suivent et qui ne conviennent pas.

VII : Les boucles

Qu'est-ce qu'une boucle ? C'est une structure qui fonctionne sur le même principe que les conditions (if... else). Concrètement, une boucle permet de répéter des instructions plusieurs fois. En clair, c'est un gain de temps, c'est très pratique, et bien souvent indispensable.

1. Une boucle simple : while

Voici comment faire avec une boucle simple : while. On va supposer que vous avez été punis et que vous devez recopier 100 fois « Je ne dois pas regarder les mouches voler quand j'apprends le PHP. ».

```
<?php
    $nombre_de_lignes = 1;
    while ($nombre_de_lignes <= 100)
    {
        echo 'Je ne dois pas regarder les mouches voler quand j'apprends le PHP.<br />';
        $nombre_de_lignes++; // $nombre_de_lignes = $nombre_de_lignes + 1
    }
?>
```

while peut se traduire par « tant que ». Ici, on demande à PHP : **TANT QUE \$nombre_de_lignes est inférieur ou égal à 100 , exécuter ces instructions.**

2. Une boucle plus complexe : for

for est un autre type de boucle, dans une forme un peu plus condensée et plus commode à écrire, ce qui fait que for est assez fréquemment utilisé. Cependant, sachez que for et while donnent le même résultat et servent à la même chose : répéter des instructions en boucle. Alors, comment ça marche un for ? Ça ressemble beaucoup au while, mais c'est la première ligne qui est un peu particulière.

```
<?php
```

```
    for ($nombre_de_lignes = 1; $nombre_de_lignes <= 100; $nombre_de_lignes++)
    {
        echo 'Ceci est la ligne n°' . $nombre_de_lignes . '<br />';
    }
```

```
?>
```

Après le mot for, il y a des parenthèses qui contiennent trois éléments, séparés par des points-virgules;

- Le premier sert à l'**initialisation**. C'est la valeur que l'on donne au départ à la variable (ici, elle vaut 1).
- Le second, c'est la **condition**. Comme pour le while, tant que la condition est remplie, la boucle est ré exécutée. Dès que la condition ne l'est plus, on en sort.
- Enfin, le troisième c'est l'**incrémentation**, qui vous permet d'ajouter 1 à la variable à chaque tour de boucle.

Les deux derniers codes donnent donc exactement le même résultat. Le for fait la même chose que le while, mais rassemble sur une seule ligne tout ce qu'il faut savoir sur le fonctionnement de la boucle.

Comment savoir lequel prendre quand je dois choisir entre un while et un for ?

La boucle while est plus simple et plus flexible : on peut faire tous les types de boucles avec mais on peut oublier de faire certaines étapes comme l'incrémentement de la variable.

En revanche, for est bien adapté quand on doit compter le nombre de fois que l'on répète les instructions et il permet de ne pas oublier de faire l'incrémentement pour augmenter la valeur de la variable !

Si vous hésitez entre les deux, il suffit simplement de vous poser la question suivante :
« **Est-ce que je sais d'avance combien de fois je veux que mes instructions soient répétées ?** ».

Si la réponse est oui, alors la boucle for est tout indiquée. Sinon, alors il vaut mieux utiliser la boucle while.

VIII : Les fonctions

En PHP, on n'aime pas avoir à répéter le même code. Pour pallier ce problème, nous avons découvert les boucles, qui permettent d'exécuter des instructions un certain nombre de fois. Ici nous allons découvrir un autre type de structure indispensable pour la suite : les fonctions. Comme les boucles, les fonctions permettent d'éviter d'avoir à répéter du code PHP que l'on utilise souvent. Mais alors que les boucles sont de bêtes machines tout juste capables de répéter deux cents fois la même chose, les fonctions sont des robots « intelligents » qui s'adaptent en fonction de ce que vous voulez faire et qui automatisent grandement la plupart des tâches courantes.

1. Qu'est ce qu'une fonction ?

Une fonction est une série d'instructions qui effectue des actions et qui retourne une valeur. En général, dès que vous avez besoin d'effectuer des opérations un peu longues dont vous aurez à nouveau besoin plus tard, il est conseillé de vérifier s'il n'existe pas déjà une fonction qui fait cela pour vous. Et si la fonction n'existe pas, vous avez la possibilité de la créer. Imaginez que les fonctions sont comme des robots.



Vous ne savez pas ce qui se passe à l'intérieur de ce robot, mais vous pouvez appuyer sur un bouton pour lui demander de faire quelque chose de précis. Avec les fonctions, c'est le même principe !

Dialogue avec une fonction

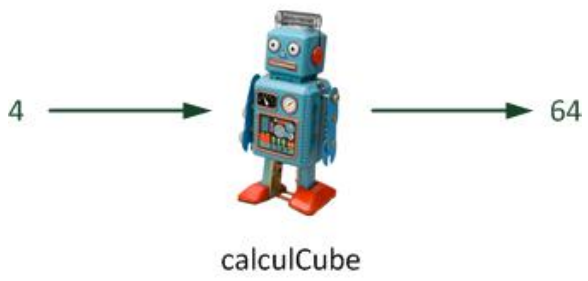
Voici le genre de dialogue qu'on peut avoir avec une fonction :

Toi, la fonction calculCube, donne-moi le volume d'un cube dont l'arête mesure 4 cm.

La fonction effectue les calculs demandés puis répond :

Ce cube a un volume de 64 cm³

On donne en **entrée** à la fonction une information appelée **paramètre** sur laquelle elle va faire des calculs (ici, la longueur de l'arête : 4) et la fonction nous retourne en **sortie** le résultat : 64. Voyez la figure suivante.



Les fonctions en PHP

Nous avons jusqu'ici imaginé le dialogue avec une fonction représentée par un robot, ce qui n'est pas très intéressant.

Revenons aux choses sérieuses et concrètes.

Appeler une fonction

En PHP, comment appelle-t-on une fonction ? Par son nom ! Par exemple :

```
<?php
    calculCube();
?>
```

Comme vous le voyez, nous avons simplement écrit le nom de la fonction, suivi de parenthèses vides, puis de l'inévitable point-virgule. En faisant cela, on appelle la fonction `calculCube` mais on ne lui envoie aucune information, aucun **paramètre**. Certaines fonctions peuvent fonctionner sans paramètres, mais elles sont assez rares. Dans le cas de `calculCube`, ça n'a pas de sens de l'appeler sans lui donner la longueur de l'arête du cube pour faire le calcul !

Si on veut lui envoyer un paramètre (un nombre, une chaîne de caractères, un booléen...), il faut l'écrire entre les parenthèses :

```
<?php
    calculCube(4);
?>
```

Ainsi, `calculCube` saura qu'elle doit travailler avec le nombre 4. Souvent, les fonctions acceptent plusieurs paramètres. Vous devez dans ce cas les séparer par des virgules :

```
<?php
    fonctionImaginaire(17, 'Vert', true, 41.7);
?>
```

Cette fonction recevra quatre paramètres : 17, le texte « Vert », le booléen vrai et le nombre 41,7.

Récupérer la valeur de retour de la fonction

Maintenant que nous savons appeler une fonction et même lui envoyer plusieurs paramètres, il faut récupérer ce qu'elle nous retourne si toutefois elle retourne quelque chose. Il y a en effet deux types de fonctions :

- celles qui ne retournent aucune valeur (ça ne les empêche pas d'effectuer des actions) ;
- celles qui retournent une valeur.

Si la fonction ne retourne aucune valeur, il n'y a rien de plus à faire que dans les codes précédents. La fonction est appelée, fait son travail, et on ne lui demande rien de plus. En revanche, si la fonction retourne une valeur (comme ça devrait être le cas pour `calculCube`), on la récupère dans une variable, comme ceci :

```
<?php
    $volume = calculCube(4);
?>
```

Sur une ligne comme celle-ci, il se passe en fait les deux choses suivantes (dans l'ordre, et de droite à gauche) :

- la fonction `calculCube` est appelée avec le paramètre 4 ;
- le résultat renvoyé par la fonction (lorsqu'elle a terminé) est stocké dans la variable `$volume`.

La variable `$volume` aura donc pour valeur 64 après l'exécution de cette ligne de code ! 44

2. Créer ses propres fonctions

Bien que PHP propose des centaines de fonctions, parfois il n'y aura pas ce que vous cherchez et il faudra écrire vous-mêmes la fonction. C'est une façon pratique d'étendre les possibilités offertes par PHP.

Quand écrire une fonction ? En général, si vous effectuez des opérations un peu complexes que vous pensez avoir besoin de refaire régulièrement, il est conseillé de créer une fonction.

Nous allons découvrir la création de fonctions à travers deux exemples :

- l'affichage d'un message de bienvenue en fonction du nom ;
- le calcul du volume d'un cône.

Premier exemple : dis bonjour au Monsieur

```
<?php
function DireBonjour($nom)
{
    echo 'Bonjour ' . $nom . ' !<br />';
}
DireBonjour('Marie');
DireBonjour('Patrice');
DireBonjour('Edouard');
?>
```

Deuxième exemple : calculer le volume d'un cône

La fonction `DireBonjour` que l'on a créée ne renvoyait aucune valeur, elle se contentait d'effectuer des actions (afficher un texte, dans le cas présent). Maintenant, nous allons créer une fonction qui renvoie une valeur.

Ici notre fonction va servir à faire un calcul : le calcul du volume d'un cône. Le principe est le suivant : vous donnez à la fonction le rayon et la hauteur du cône, elle travaille et vous renvoie le volume que vous cherchiez.

Formule : Il faut connaître le rayon de la base et la hauteur. La formule à utiliser pour trouver le volume est : $rayon^2 \times \pi \times hauteur \times 1/3$.

```
<?php
function VolumeCone($rayon, $hauteur)
{
    $volume = $rayon * $rayon * 3.14 * $hauteur * (1/3); // calcul du volume
    return $volume; // indique la valeur à renvoyer, ici le volume
}
```

```
$volume = VolumeCone(3, 1);
echo 'Le volume d\'un cône de rayon 3 et de hauteur 1 est de ' . $volume;
?>
```

IX : Les tableaux

1. Les deux types de tableaux

Un tableau (aussi appelé **array**) est une variable. Mais une variable un peu spéciale. Jusqu'ici vous avez travaillé avec des variables toutes simples : elles ont un nom et une valeur.

Ici, nous allons voir qu'il est possible d'enregistrer de nombreuses informations dans une seule variable grâce aux tableaux. On en distingue deux types :

- les tableaux numérotés ;
- les tableaux associatifs.

Les tableaux numérotés

Ces tableaux sont très simples à imaginer. Regardez par exemple celui-ci, contenu de la variable \$prenoms :

Clé	Valeur
0	François
1	Michel
2	Nicole
3	Véronique
4	Benoît
...	...

\$prenoms est un **array** : c'est ce qu'on appelle une variable « tableau ». Elle n'a pas qu'une valeur, mais plusieurs (vous pouvez en mettre autant que vous voulez). Dans un array, les valeurs sont rangées dans des « cases » différentes. Ici, nous travaillons sur un array *numéroté*, c'est-à-dire que chaque case est identifiée par un numéro. Ce numéro est appelé **clé**.

Construire un tableau numéroté

Pour créer un tableau numéroté en PHP, on utilise généralement la fonction array. Cet exemple vous montre comment créer l'array \$prenoms :

```
<?php
    $prenoms = array ('François', 'Michel', 'Nicole', 'Véronique', 'Benoît');
?>
```

L'ordre a beaucoup d'importance. Le premier élément (« François ») aura le n°0, ensuite Michel le n°1, etc.

Vous pouvez aussi créer manuellement le tableau case par case :

```
<?php
    $prenoms[0] = 'François';
    $prenoms[1] = 'Michel';
    $prenoms[2] = 'Nicole';
?>
```


Si vous ne voulez pas avoir à écrire vous-mêmes le numéro de la case que vous créez, vous pouvez laisser PHP le sélectionner automatiquement en laissant les crochets vides :

```
<?php
    $prenoms[] = 'François'; // Créera $prenoms[0]
    $prenoms[] = 'Michel'; // Créera $prenoms[1]
    $prenoms[] = 'Nicole'; // Créera $prenoms[2]
?>
```

Afficher un tableau numéroté

Pour afficher un élément, il faut donner sa position entre crochets après \$prenoms. Cela revient à dire à PHP :

« Affiche-moi le contenu de la case n°1 de \$prenoms. »

Pour faire cela en PHP, il faut écrire le nom de la variable, suivi du numéro entre crochets. Pour afficher « Michel », on doit donc écrire :

```
<?php
    echo $prenoms[1];
?>
```

Les tableaux associatifs

Les tableaux associatifs fonctionnent sur le même principe, sauf qu'au lieu de numéroter les cases, on va les étiqueter en leur donnant à chacune un nom différent.

Par exemple, supposons que je veuille, dans un seul array, enregistrer les coordonnées de quelqu'un (nom, prénom, adresse, ville, etc.). Si l'array est numéroté, comment savoir que le n°0 est le nom, le n°1 le prénom, le n°2 l'adresse... ? C'est là que les tableaux associatifs deviennent utiles.

Construire un tableau associatif

Pour en créer un, on utilisera la fonction array comme tout à l'heure, mais on va mettre « l'étiquette » devant chaque information :

```
<?php
    $coordonnees = array (
        'prenom' => 'François',
        'nom' => 'Dupont',
        'adresse' => '3 Rue du Paradis',
        'ville' => 'Marseille');
?>
```

Vous remarquez qu'on écrit une flèche (=>) pour dire « associé à ». Par exemple, on dit « *ville* » associée à « *Marseille* ».

Nous avons créé un tableau qui ressemble à la structure suivante :

Clé	Valeur
prenom	François
nom	Dupont
adresse	3 Rue du Paradis
ville	Marseille

Il est aussi possible de créer le tableau case par case, comme ceci :

```
<?php
    $coordonnees['prenom'] = 'François';
    $coordonnees['nom'] = 'Dupont';
    $coordonnees['adresse'] = '3 Rue du Paradis';
    $coordonnees['ville'] = 'Marseille';
?>
```

Afficher un tableau associatif

Pour afficher un élément, il suffit d'indiquer le nom de cet élément entre crochets, ainsi qu'entre guillemets ou apostrophes puisque l'étiquette du tableau associatif est un texte. Par exemple, pour extraire la ville, on devra taper :

```
<?php
    echo $coordonnees['ville']; //Affichera Marseille
?>
```

2. Parcourir un tableau

Lorsqu'un tableau a été créé, on a souvent besoin de le parcourir pour savoir ce qu'il contient. Nous allons voir trois moyens d'explorer un array :

- la boucle for ;
- la boucle foreach ;
- la fonction print_r (utilisée principalement pour le débogage).

La boucle for

Il est très simple de parcourir un tableau numéroté avec une boucle for. En effet, puisqu'il est numéroté à partir de 0, on peut faire une boucle for qui incrémente un compteur à partir de 0 :

```
<?php
// On crée notre array $prenoms
$prenoms = array ('François', 'Michel', 'Nicole', 'Véronique', 'Benoît');

// Puis on fait une boucle pour tout afficher :
for ($numero = 0; $numero < 5; $numero++)
{
    echo $prenoms[$numero] . '<br />'; // affichera $prenoms[0], $prenoms[1] etc.
}
?>
```

La boucle foreach

La boucle for a beau fonctionner, on peut utiliser un autre type de boucle plus adapté aux tableaux qu'on n'a pas encore vu jusqu'ici : **foreach**. C'est une sorte de boucle for spécialisée dans les tableaux.

foreach va passer en revue chaque ligne du tableau, et lors de chaque passage, elle va mettre la valeur de cette ligne dans une variable temporaire (par exemple \$element).

```
<?php
$prenoms = array ('François', 'Michel', 'Nicole', 'Véronique', 'Benoît');

foreach($prenoms as $element)
{
    echo $element . '<br />'; // affichera $prenoms[0], $prenoms[1] etc.
}
?>
```

C'est le même code que tout à l'heure mais cette fois basé sur une boucle foreach. À chaque tour de boucle, la valeur de l'élément suivant est mise dans la variable \$element. On peut donc utiliser \$element uniquement à l'intérieur de la boucle afin d'afficher l'élément en cours.

L'avantage de foreach est qu'il permet aussi de parcourir les tableaux associatifs.

```
<?php
$coordonnees = array (
    'prenom' => 'François',
    'nom' => 'Dupont',
    'adresse' => '3 Rue du Paradis',
    'ville' => 'Marseille');

foreach($coordonnees as $element)
{
    echo $element . '<br />';
}
?>
```

foreach va mettre tour à tour dans la variable `$element` le prénom, le nom, l'adresse et la ville contenus dans l'array `$coordonnees`.

On met donc entre parenthèses :

d'abord le nom de l'array (ici `$coordonnees`) ;

ensuite le mot-clé `as` (qui signifie quelque chose comme « en tant que ») ;

enfin, le nom d'une variable que vous choisissiez et qui va contenir tour à tour chacun des éléments de l'array (ici `$element`).

Entre les accolades, on n'utilise donc que la variable `$element`.

La boucle s'arrête lorsqu'on a parcouru tous les éléments de l'array.

Toutefois, avec cet exemple, on ne récupère que la valeur. Or, on peut aussi récupérer la clé de l'élément. On doit dans ce cas écrire foreach comme ceci :

```
<?php foreach($coordonnees as $cle => $element) ?>
```

À chaque tour de boucle, on disposera non pas d'une, mais de deux variables :

\$cle, qui contiendra la clé de l'élément en cours d'analyse (« prenom », « nom », etc.) ;

\$element, qui contiendra la valeur de l'élément en cours (« François », « Dupont », etc.).

Testons le fonctionnement avec un exemple :

```
<?php
$coordonnees = array (
    'prenom' => 'François',
    'nom' => 'Dupont',
    'adresse' => '3 Rue du Paradis',
    'ville' => 'Marseille');

foreach($coordonnees as $cle => $element)
{
    echo '[' . $cle . '] vaut ' . $element . '<br />';
}
?>
```

Avec cette façon de procéder, vous avez maintenant dans la boucle la clé ET la valeur.

Afficher rapidement un array avec print_r

Parfois, en codant votre site en PHP, vous aurez sous les bras un array et vous voudrez savoir ce qu'il contient, juste pour votre information. Vous pourriez utiliser une boucle for ou, mieux, une boucle foreach. Mais si vous n'avez pas besoin d'une mise en forme spéciale et que vous voulez juste savoir ce que contient l'array, vous pouvez faire appel à la fonction print_r. C'est une sorte de echo spécialisé dans les arrays.

Cette commande a toutefois un défaut : elle ne renvoie pas de code HTML comme `<br />` pour les retours à la ligne. Pour bien les voir, il faut donc utiliser la balise HTML `<pre>` qui nous permet d'avoir un affichage plus correct.

```
<?php
$coordonnees = array (
    'prenom' => 'François',
    'nom' => 'Dupont',
    'adresse' => '3 Rue du Paradis',
    'ville' => 'Marseille');

echo '<pre>';
print_r($coordonnees);
echo '</pre>';
?>
```